Comprehensive Development Plan & Ticket Backlog: PBI MCP Project

Status: Discovery Phase Complete / Architecture Defined **Project Goal:** Build a standalone Model Context Protocol (MCP) server for Power BI authoring and editing that works exclusively with local .pbip files (no Microsoft Cloud or Azure Auth required).

1. Core Architecture Concept

The server implements a **Dual-Engine Architecture** to handle the two distinct data formats within a .pbip project:

Engine A: Semantic Model (The "Brain")

- **Format:** TMDL (Tabular Model Definition Language).
- **Primary Tool:** Atzingen/tmdl-parser (Python).
- **Function:** Parses, manipulates, and serializes the semantic model (tables, measures, DAX, relationships).
- **Approach:** Treat the model as a graph of Pythonic objects.

Engine B: Report Layout (The "Body")

- **Format:** Deeply nested JSON files.
- **Primary Tools:** Python json module + jmespath (for complex querying).
- **Function:** Manages visual placement, colors, slicers, and visual-to-model mappings.
- **Approach:** Implement a **"Shadow Model"** pattern to abstract verbose JSON into a simplified, flat structure for the agent to manipulate.

2. Technical Requirements & Constraints

- **Standalone:** Must function entirely offline/locally.
- **File System:** Direct interaction with .pbip directory structures.
- **Data Integrity (Critical):** Any change to the Semantic Model (e.g., renaming a measure in TMDL) must trigger a reconciliation process in the JSON Layer to update all corresponding string references in the report layouts safely.
- **Complexity Management:** Use a "Path-based" interface for visual properties to prevent the agent from needing to navigate deep JSON nesting.

3. Containerization Specification

To ensure the MCP server runs reliably and can be easily deployed locally or linked to desktop clients via standard input/output (stdio), we use a containerized approach with Docker.

Base Image

- **python:3.11-slim**: Chosen for its minimal footprint, security profile, and native compatibility with Python-based parsers and MCP libraries.

Dockerfile

```
# syntax=docker/dockerfile:1
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    git \
    && rm -rf /var/lib/apt/lists/*

# Install Python package management tools
RUN pip install --no-cache-dir uv

# Copy dependency definitions
COPY pyproject.toml uv.lock* ./

# Install project dependencies
RUN uv config virtualenvs.create false \
    && uv install --no-interaction --no-ansi --no-root

# Copy the rest of the application code
COPY . .

# Install the package itself
RUN uv install --no-interaction --no-ansi

# Default command to run the MCP server
ENTRYPOINT ["python", "-m", "mcp_server.main"]
```

docker-compose.yml

```
version: '3.8'

services:
  pbi-mcp-server:
    build: .
    image: pbi-mcp-server:latest
    container_name: pbi-mcp-server
```

```
volumes:
  - ./sample_project.pbip:/app/workspace:rw
environment:
  - PBI_WORKSPACE_PATH=/app/workspace
  - LOG_LEVEL=INFO
stdin_open: true
tty: true
```

4. Jira / ADO Ticket Backlog

Epic 1: Project Foundation & Environment Setup

- **PBIP-101: Initialize Repository & Dependency Management**
  - **Type:** Task
  - **Description:** Set up the Python project structure using uv. Add core dependencies (mcp, pydantic, jmespath, tmdl-parser). Define .gitignore for working files and caches.
- **PBIP-102: TMDL Round-Trip Fidelity Spike**
  - **Type:** Spike / R&D
  - **Description:** Write a spike test script loading a sample .tmdl table file, parsing it, and serializing it back to disk.
  - **Acceptance Criteria:** Run git diff on output. Indentation, whitespace, and developer comments must remain intact to prevent noisy diffs.
- **PBIP-103: In-Memory State Caching Layer**
  - **Type:** Story
  - **Description:** Implement a file-system state cache wrapper using file modification timestamps (os.path.getmtime) to avoid redundant parsing across agent turns.

Epic 2: Engine A — Semantic Model (TMDL)

- **PBIP-201: TMDL Parser Wrapper Implementation**
  - **Type:** Story
  - **Description:** Build an abstraction class around tmdl-parser to ingest definition folders and expose tables, columns, and measures as typed Pydantic models.
- **PBIP-202: Semantic Model Read Operations (MCP Tools)**
  - **Type:** Story
  - **Description:** Implement MCP tool endpoints to expose model summaries (listing tables, measures, and calculated columns) cleanly without flooding context windows.
- **PBIP-203: Semantic Model Write Operations**
  - **Type:** Story
  - **Description:** Implement structured write methods for updating measure DAX expressions (update_measure_dax()) and renaming measures via Engine A.

Epic 3: Engine B — Report Layout (JSON Shadow Model)

- **PBIP-301: JSON Layout Flattener (Shadow Model Pattern)**
  - **Type:** Story
  - **Description:** Build a parser utility utilizing Python's json module and jmespath to ingest report layouts and expose them via a flat, path-based addressing scheme.
- **PBIP-302: Visual Property Query Tools (MCP)**
  - **Type:** Story
  - **Description:** Create targeted MCP tool wrappers allowing the LLM to inspect visual properties, slicers, and layout properties using path syntax.

Epic 4: Cross-Layer Integration & Integrity

- **PBIP-401: Reference-Aware Rename Engine**
  - **Type:** Story
  - **Description:** Build an intelligent dependency mapper scanning report layout JSON properties for bound identifiers. Ensure renames avoid destructive naive substring string-replacement.
- **PBIP-402: Integration Test — The "Rename Measure" Workflow**
  - **Type:** Test / QA
  - **Description:** Build an end-to-end integration test executing a full rename workflow (Total Revenue \rightarrow Gross Revenue): updating TMDL definitions, updating JSON layout references, and verifying file consistency.
- **PBIP-403: Power BI Structure Validation Safety Net**
  - **Type:** Story
  - **Description:** Write a post-write syntax validator checking modified .pbip directories for structural correctness before releasing control back to the user.

Epic 5: Containerization & Local Deployment

- **PBIP-501: Create Base Dockerfile**
  - **Type:** Task
  - **Description:** Create a production-ready Dockerfile using python:3.11-slim as the base image, installing system dependencies (git), uv, and configuring stdio execution for the MCP server.
- **PBIP-502: Configure Docker Compose for Local Volume Mounting**
  - **Type:** Story
  - **Description:** Create a docker-compose.yml file to orchestrate the container with volume mapping for local .pbip directories (/app/workspace).
- **PBIP-503: Container Integration Testing**
  - **Type:** Test / QA
  - **Description:** Verify that the MCP server runs successfully inside the Docker container, responds to client initialization requests over stdio, and correctly handles file reads/writes on mounted volumes.